

Verilog

- **Synthesis**: describe hardware that you ultimately want to create
- **Simulation**: describe hardware for simulation, and describe tests

Synthesizable Language Constructs

- Entities & Architecture $\rightarrow$ Modules
- Signals $\rightarrow$ Wires & Regs
- Concurrent Signal Assignments
- Module Instantiations
- Processes $\rightarrow$ Always Blocks Each Process is synthesized into a concurrent event
 - if/else Conditional Statement
 - Case Statement

Synthesis tools use **pattern matching** for each process

- any process must follow one of three patterns:
 1. **Purely Combinational**
 - every input to process must be in sensitivity list
 - every output must be assigned a value for every possible combination of inputs
 2. **Sequential**
 - only clock should be in sensitivity list
 - only signals that change on **same edge** of **same clock** should be part of block
 3. **Sequential with Asynchronous Reset**
 - sensitivity list includes clock and reset
- **Asynchronous Reset**: flip-flop is set to 0 upon reset, regardless of clock

Netlist

- using synthesis tools, generate a netlist that imitates programmed FPGA hardware
- can be used to see what written Verilog will actually synthesize to

Behavioural For Loop

- only inside always/process block
- must have **predetermined length** to be synthesizable
- can be used to replace repetitive code

Generate Statements

- shorthand for repetitive instantiation

Ex: 4 mux instantiations

generate i;

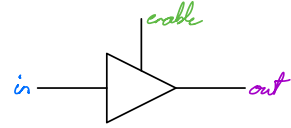
generate

for (i=0; i<4; i=i+1) begin: **big_mux** Generate name required
4mux mux (w[4*i], w[4*i+1], w[4*i+2], w[4*i+3], s[i:0], m[i]);

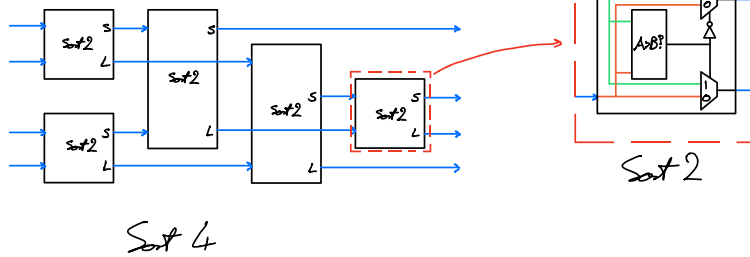
end

endgenerate

- can use if statements inside generate statements for ambiguous situations
- Generate if & for are *concurrent*
- Behavioural if & for are *sequential*
- Tri-State logic
 - if *enable* is 1, then *out* is driven by *in*
 - if *enable* is 0, then *out* is not driven (i.e. high impedance $\rightarrow Z$)



- Sorting
 - Combinational: Big & Fast

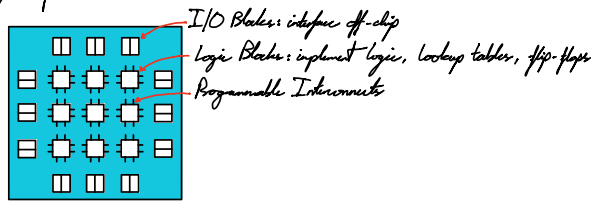


- Amount of logic grows quickly as sorting count increases
- Cannot be used for undetermined sorting capacity
- Sequential: Small & Slow
 - Constant *datapath* to sort over multiple clock cycles (state machine)

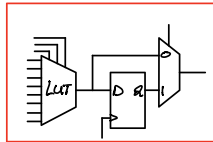
Optimization & FPGA Architectures

- Fully Custom Integrated Circuit (ASIC)
 - best density and speed
 - difficult and time-consuming to design
- Standard Cell Design
 - library of cells that implement different gates
- Field-Programmable Gate Arrays
 - quick designing/testing/manufacturing
- FPGA Compilation
 1. Synthesis: associates RTL with gates
 2. Placement: assign logic blocks to specific chip location
 3. Routing: connections between design components are made
 4. Timing Analysis: checks if timing constraints were passed to implementation

• Design of FPGA



- each **always/process** block is compiled as a separate hardware unit that operates in parallel
- **Lookup Table**: replace runtime computation with simple array indexing operation.
- **Logic Block**



- this block allows a **combinational function** and/or a **flip-flop**

- Placement has many solutions → best solution results in fastest clock speed
 - Heuristic Algorithm using an **iterative** approach can find a good solution
- Routing
 - **Switch Blocks**: connect horizontal and vertical channels inbetween logic and I/O blocks ^{made of metal tracks}
 - using every possible connection is big and slow → special **topologies** are used to maximize efficiency
 - new FPGAs support **hyper-pipelining** by including flip-flops at every switch
- Timing Analysis
 - Find delay of all paths
 - Determine critical path: looks for timing violations
- Embedded Processors
 - Soft: implemented using FPGA resources
 - Hard: embedded into FPGA

— On-Chip Memory

- Embedded Memory Blocks
 - built into FPGA
 - good for large storage
 - size/bit width are limited → less flexible storage
 - Single-Port Memory Configuration
 - each cycle → read OR write
 - Dual-Port Memory Configuration
 - each cycle perform 2 operations → read OR write

- **Flip-Flops**
 - uses flip-flops contained within logic blocks
 - fast, at the expense of FPGA resources
 - **Port Configuration**
 - Read: use mux to select address
 - Write: decoder with "n" outputs
- **Configuration Bits**
 - uses configuration bits contained within logic blocks
- **Including Memories in Verilog Design**
 - **Explicit Instantiation**
 - explicitly indicate memory usage
 - **Inferred Memories**
 - write memory behaviour $\rightarrow$ usually synthesizes to flip-flops over embedded memory
- **Scheduling**
 - availability of memory ports requires scheduling to avoid timing errors
 - 1. **Embedded Memory**: create schedule that meets constraints on ports
 - 2. **Flip-Flop Memory**: any schedule works $\rightarrow$ synthesis tool determines number of ports

Timing

- **performance** and **power** are the two optimization goals of a circuit
- **Parasitic Capacitance**: unwanted capacitance that exists between mosfets/transistors
- the **delay** of a logic gate is the time to charge the parasitic capacitance of its output
 - delay of n gates is proportional to $n \cdot R \cdot C$
 - **resistance** depends on:
 - length of wire (longer $\rightarrow$ higher)
 - size of transistor (bigger $\rightarrow$ lower)
 - **capacitance** depends on:
 - length of wire (longer $\rightarrow$ higher)
 - size of transistor (bigger $\rightarrow$ higher)
 - number of gates driven $\rightarrow$ **fanout** $\leftarrow$ one gate drives multiple gates
- **Combinational Timing Constraints**
 - **Gate Propagation Delay**: time for combinational logic output to change after input changes
 - **Critical Path Delay**: time for output of series of gates to change after input changes
 - this limits the **clock speed**
 - **Critical Path**: path through combinational logic that takes the longest
- **Fan-Out**
 - single gate drives multiple gates; greater total **parasitic capacitance**
 - tell compiler to increase number of driving gates

Flip-Flop Timing Constraints

- Clock-to-Q Delay: time for output of flip-flop to settle after clock rising-edge occurs
- Setup time: time for input to propagate *into* flip-flop
- Hold time: time for flip-flop to read input before value changes

Setup Violation

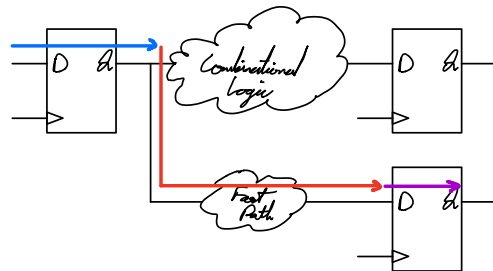


- time for output to settle
- time through critical path
- value arrives early enough for setup time

$$t_{\text{clock}} \geq t_{\text{clk-to-q, max}} + t_{\text{critical path}} + t_{\text{setup}}$$

- Fix: slow clock / reduce critical path length

Hold Violation



- flip-flop's output settles
- fast path settles very quickly
- destination flop is reading old value

$$t_{\text{hold}} \leq t_{\text{clk-to-q, min}} + t_{\text{fast path}}$$

- Fix: add more gate delays

Timing Closure: ensuring design meets timing constraints

- Slow Clock Frequency: smaller circuit $\rightarrow$ longer & less paths
- Fast Clock Frequency: larger circuit $\rightarrow$ shorter & more paths

Pipelining

- separate long combinational paths by inserting flip-flops
- Retiming: balance out path delay by moving gates before/after flip-flops
- Clock Skew: the same clock signal arrives at different components at different times created by:
 - temperature of wires
 - difference in clock-signal travel distance
 - physical composition
- can be *decreased* through symmetrical layout $\rightarrow$ decrease path differences
- Phase-Locked Loop
 - generates output whose *phase* is related to phase of input

- **Glitches**: undesired short-lived pulse that occurs before a signal settles to intended value
created by:
 - unequal arrival times of inputs on combinational gates
 - can avoid by assigning outputs to state **encoding bits** or by adding synchronous **flip-flop** before output

System on Chip

- combines various circuits onto a single chip
- **Hardware Accelerator**: hardware that performs specific function more efficiently
- SoCs are good for high-complexity, low-performance tasks

composed of:

1. CPU

- **Hardware**: built in CPU $\rightarrow$ outside of FPGA fabric
- **Software**: implemented through logic synthesis

2. Interconnect

- provide routing paths to connect inputs and outputs of components
- implemented using wires, multiplexers, arbiters
- arbitration requires **flow control**
- **Inter-Module Protocol Description**
 - flow control mechanism $\rightarrow$ issues requests / returns results / handle multiple requests
- **Memory Maps**: each **device** has its own base address that is used by interconnect

• address breakdown:

31:9	8:2	1:0
device identification	address	byte select

Metastability

- the ability of a system to persist for an unbounded time in an unstable equilibrium

$$\text{Mean Time Between Failure} = \frac{e^{\frac{t_{\text{setup}}}{t_0}}}{C_1 \cdot f_{\text{clk}} \cdot t_{\text{hold}}}$$

t_{setup} slack available for metastable solution

- larger slack (t_{hold}) results in larger MTBF

Power

- **Dynamic Power**: everytime a wire switches from 0 to 1 or 1 to 0 power is dissipated

$$P_{\text{dynamic}} = \alpha \cdot f \cdot C \cdot V^2$$

- **Static Power**: all transistors on chip leak power
- **Clock Gating**: disable clock signal to unused FPGA components

State Machines

- Synchronous FSM
 - transitions state on *clock edge*
- Asynchronous FSM
 - transitions state *as soon as possible*
- Stability: stable when for any next-state logic input values, the circuit reaches and remains in another state
- State Encoding Assignment in Asynchronous
 - Hamming Distance of encoding between adjacent states must be 1
 - can go to wrong state during transition

000 $\xrightarrow{\text{green}}$ 010 $\rightarrow$ Hamming Distance of 1
000 $\xrightarrow{\text{red}}$ 011

Datapaths

- Synchronous Datapath
 - *clock* signal coordinates the flow of data
- Asynchronous Datapath
 - *each stage* coordinates with other stages for moving data
 - 4 phase Handshaking Protocol
 1. sender issues data; *RDY* = High
 2. receiver reads data; *ACK* = High
 3. sender accepts ACK; *RDY* = Low
 4. receiver; *ACK* = Low
- Data Encoding Scheme
 - Single Rail: one wire for each bit
 - Dual Rail: two wires; one for bit, one for ready signal